

Lab Assignment 2

Name: **Fulekar Hitali Sameer**

Roll No.: **23103051**

CSE 6TH SEM (6A G2)

Lab Assignment 2

Machine Learning Lab - CSDC0336

(NITJ-CSED)

Learning goals:

- Load and manipulate real-world numerical datasets using NumPy.
- Replace manual Python loops with high-performance vectorized operations.
- Perform advanced preprocessing including statistical imputation and logical filtering.

Task 1: Efficient Sorting

Load the first column (Sepal Length) of the Iris dataset into a NumPy array. In Lab 1, you used a while loop to sort a small list. Now, use NumPy's built-in sorting function to sort all 150 sepal lengths in ascending order. Write a brief comment in your Colab cell comparing the effort of writing a manual loop versus using NumPy.

Task 2: Scalable Data Simulation

In ML, we often need to generate "dummy data" to test algorithms. Use NumPy to generate a 1000 x 4 matrix of random floating-point numbers (simulating 1000 new flower samples). Calculate the Mean and Standard Deviation of each column in this matrix using NumPy's axis parameter, without using any for loops.

Task 3: Intelligent Data Filtering

Filtering is essential for cleaning data. From the Iris dataset, use Boolean Indexing to create a new array called `large_petal` that contains only the rows where the Petal Length (Column 3) is greater than 4.0. Ensure no if statements are used in your code.

Task 4: Categorical Encoding

Suppose you have a NumPy array of the flower species: `species = np.array(['Setosa', 'Versicolor', 'Virginica', 'Setosa'])`. Use the `np.where` function to create a numeric "Mask" where 'Setosa' is encoded as 1 and all other species are encoded as 0.

Task 5: Handling "Missing" Data in Real Arrays

In real-world data, some measurements might be missing. Create a copy of your Iris array and manually insert `np.nan` at index [0,0]. Then:

- Use `np.isnan()` to locate the missing value.
- Replace that nan value with the Mean of that specific column using `np.nanmean()`.

Dataset Link: - <https://www.kaggle.com/datasets/arshid/iris-flower-dataset>

Task 1: Efficient Sorting Load the first column (Sepal Length) of the Iris dataset into a NumPy array. In Lab 1, you used a while loop to sort a small list. Now, use NumPy's built-in sorting function to sort all 150 sepal lengths in ascending order. Write a brief comment in your Colab cell comparing the effort of writing a manual loop versus using NumPy.

```
import numpy as np
```

```
import pandas as pd

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"

columns = ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Species']

df = pd.read_csv(url, header=None, names=columns)

df.head()
```

```
print("hitali , 23103051")
```

```
hitali , 23103051
```

```
sepal_length = df['SepalLengthCm'].values
```

```
sorted_sepal = np.sort(sepal_length)
```

```
print("Sorted Sepal Lengths:")
```

```
print(sorted_sepal)
```

```
print("hitali , 23103051")
```

```
Sorted Sepal Lengths:
```

```
[4.3 4.4 4.4 4.4 4.5 4.6 4.6 4.6 4.6 4.7 4.7 4.8 4.8 4.8 4.8 4.8 4.9 4.9
 4.9 4.9 4.9 4.9 5. 5. 5. 5. 5. 5. 5. 5. 5.1 5.1 5.1 5.1
 5.1 5.1 5.1 5.1 5.1 5.2 5.2 5.2 5.2 5.2 5.3 5.4 5.4 5.4 5.4 5.4 5.4 5.5
 5.5 5.5 5.5 5.5 5.5 5.6 5.6 5.6 5.6 5.6 5.6 5.7 5.7 5.7 5.7 5.7 5.7 5.7
 5.7 5.8 5.8 5.8 5.8 5.8 5.8 5.8 5.9 5.9 5.9 6. 6. 6. 6. 6. 6. 6.1
 6.1 6.1 6.1 6.1 6.1 6.2 6.2 6.2 6.2 6.3 6.3 6.3 6.3 6.3 6.3 6.3 6.3
 6.4 6.4 6.4 6.4 6.4 6.4 6.4 6.5 6.5 6.5 6.5 6.5 6.5 6.6 6.6 6.7 6.7 6.7
 6.7 6.7 6.7 6.7 6.8 6.8 6.8 6.9 6.9 6.9 6.9 7. 7.1 7.2 7.2 7.2 7.3 7.4
 7.6 7.7 7.7 7.7 7.7 7.9]
```

```
hitali , 23103051
```

Task 2: Scalable Data Simulation In ML, we often need to generate "dummy data" to test algorithms. Use NumPy to generate a 1000 x 4 matrix of random floating-point numbers (simulating 1000 new flower samples). Calculate the Mean and Standard Deviation of each column in this matrix using NumPy's axis parameter, without using any for loops.

```
data = np.random.rand(1000, 4)
```

```
mean_values = np.mean(data, axis=0)
```

```
std_values = np.std(data, axis=0)
```

```
print("Column-wise Mean:", mean_values)
```

```
print("Column-wise Standard Deviation:", std_values)
```

```
print("hitali , 23103051")
```

```
Column-wise Mean: [0.51788092 0.47879375 0.48925302 0.49643492]
```

```
Column-wise Standard Deviation: [0.29122229 0.28490982 0.2886335 0.29354982]
```

```
hitali , 23103051
```

Task 3: Intelligent Data Filtering Filtering is essential for cleaning data. From the Iris dataset, use Boolean Indexing to create a new array called `large_petals` that contains only the rows where the Petal Length (Column 3) is greater than 4.0. Ensure no if statements are used in your code.

```
iris_array = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']].values
```

```
large_petals = iris_array[iris_array[:, 2] > 4.0]
```

```
print("Rows with Petal Length > 4.0:")
```

```
print(large_petals)
```

```
print("hitali , 23103051")
```

```

[6.5 3. 5.5 1.8]
[7.7 3.8 6.7 2.2]
[7.7 2.6 6.9 2.3]
[6. 2.2 5. 1.5]
[6.9 3.2 5.7 2.3]
[5.6 2.8 4.9 2. ]
[7.7 2.8 6.7 2. ]
[6.3 2.7 4.9 1.8]
[6.7 3.3 5.7 2.1]
[7.2 3.2 6. 1.8]
[6.2 2.8 4.8 1.8]
[6.1 3. 4.9 1.8]
[6.4 2.8 5.6 2.1]
[7.2 3. 5.8 1.6]
[7.4 2.8 6.1 1.9]
[7.9 3.8 6.4 2. ]
[6.4 2.8 5.6 2.2]
[6.3 2.8 5.1 1.5]
[6.1 2.6 5.6 1.4]
[7.7 3. 6.1 2.3]
[6.3 3.4 5.6 2.4]
[6.4 3.1 5.5 1.8]
[6. 3. 4.8 1.8]
[6.9 3.1 5.4 2.1]
[6.7 3.1 5.6 2.4]
[6.9 3.1 5.1 2.3]
[5.8 2.7 5.1 1.9]
[6.8 3.2 5.9 2.3]
[6.7 3.3 5.7 2.5]
[6.7 3. 5.2 2.3]
[6.3 2.5 5. 1.9]
[6.5 3. 5.2 2. ]
[6.2 3.4 5.4 2.3]
[5.9 3. 5.1 1.8]
hitali , 23103051

```

Task 4: Categorical Encoding Suppose you have a NumPy array of the flower species: `species = np.array(['Setosa', 'Versicolor', 'Virginica', 'Setosa'])`. Use the `np.where` function to create a numeric "Mask" where 'Setosa' is encoded as 1 and all other species are encoded as 0.

```

species = np.array(['Setosa', 'Versicolor', 'Virginica', 'Setosa'])

mask = np.where(species == 'Setosa', 1, 0)

print("Encoded Mask:", mask)

print("hitali , 23103051")

```

```

Encoded Mask: [1 0 0 1]
hitali , 23103051

```

Task 5: Handling "Missing" Data in Real Arrays In real-world data, some measurements might be missing. Create a copy of your Iris array and manually insert `np.nan` at index [0,0]. Then:

- Use `np.isnan()` to locate the missing value.
- Replace that nan value with the Mean of that specific column using `np.nanmean()`.

```

iris_copy = iris_array.copy()

iris_copy[0,0] = np.nan

print("Array with NaN inserted:")
print(iris_copy[:5])

print("hitali , 23103051")

print("Location of NaN:", np.isnan(iris_copy))

print("hitali , 23103051")

```



```
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
[False False False False]
```

```
col_mean = np.nanmean(iris_copy[:,0])
iris_copy[np.isnan(iris_copy)] = col_mean
print("After replacing NaN with column mean:")
print(iris_copy[:5])
print("hitali , 23103051")
```

```
After replacing NaN with column mean:
[[5.84832215 3.5      1.4      0.2      ]
 [4.9        3.       1.4      0.2      ]
 [4.7        3.2      1.3      0.2      ]
 [4.6        3.1      1.5      0.2      ]
 [5.         3.6      1.4      0.2      ]]
hitali , 23103051
```

End of Assignment